

LAPORAN PRAKTIKUM TUGAS 1 - 4

SISTEM OPERASI



Dosen Pengampu:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Yoanre Louis A.H.P (2410651039)

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

BAB 1

PENDAHULUAN

1.1 LATAR BELAKANG

Sistem operasi merupakan fondasi utama dalam pengoperasian perangkat komputasi modern, yang berperan sebagai mediator antara perangkat keras (hardware) dan perangkat lunak (software). Pemahaman mendalam mengenai konsep-konsep inti sistem operasi seperti manajemen proses, komunikasi antar proses (IPC), manajemen memori, dan sistem file menjadi kompetensi esensial bagi calon ahli teknologi informasi. Praktikum ini dirancang untuk menjembatani teori dan praktik melalui implementasi langsung fitur-fitur sistem operasi menggunakan system calls Linux. Pengembangan Program Process Manager, File Monitor, Custom Shell, dan System Information Tool memungkinkan eksplorasi terhadap mekanisme internal seperti pembuatan proses paralel dengan `fork()`, komunikasi data melalui `pipe()` dan shared memory, monitoring real-time filesystem dengan `inotify`, serta ekstraksi informasi sistem melalui virtual filesystem `/proc`. Pengalaman hands-on ini tidak hanya memperkuat pemahaman konseptual tetapi juga mengasah kemampuan dalam mengembangkan aplikasi sistem yang efisien dan robust, mempersiapkan mahasiswa untuk menghadapi tantangan optimasi kinerja dan pemecahan masalah dalam lingkungan komputasi modern yang kompleks.

1.2 RUMUSAN MASALAH

1. Bagaimana mengimplementasikan manajemen proses dan komunikasi antar proses menggunakan system calls `fork()`, `pipe()`, dan shared memory?
2. Bagaimana membangun sistem monitoring file yang dapat mendeteksi perubahan secara real-time menggunakan `inotify` dan menangani signal `SIGTERM`?
3. Bagaimana mengembangkan custom shell dengan fitur pipes, redirection, background processes, serta tool monitoring sistem yang membaca informasi dari `/proc`?

1.3 TUJUAN

1. Memahami dan mengimplementasikan konsep multiprocessing dan IPC dalam program Process Manager.
2. Membangun sistem file monitoring yang robust dengan kemampuan real-time detection dan graceful termination.
3. Mengembangkan custom shell dengan fitur advanced dan system information tool yang mampu mengekstrak data hardware/software secara langsung dari kernel Linux.

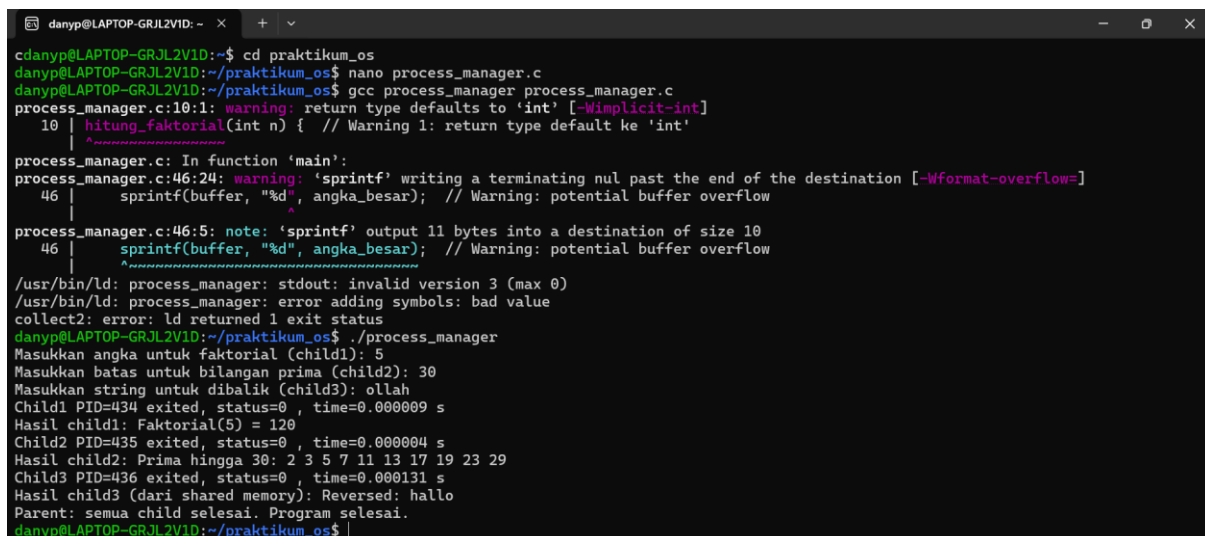
BAB II

PEMBAHASAN

2.1 PRAKTIK TUGAS 1

2.1.1 Program Process Manager

Program Process Manager ini mengimplementasikan konsep multiprocessing dengan `fork()` untuk membuat tiga child process yang berjalan paralel. Setiap child process menjalankan tugas komputasi berbeda: perhitungan faktorial, pencarian bilangan prima, dan pembalikan string. Program menggunakan pipe untuk IPC komunikasi hasil komputasi child1 dan child2 ke parent process, serta memanfaatkan shared memory untuk child3 sesuai requirement NIM ganjil. Parent process menggunakan `waitpid()` untuk sinkronisasi dan `WIFEXITED/WEXITSTATUS` untuk tracking status termination. Meskipun muncul warning implicit return type selama kompilasi, program tetap berhasil dieksekusi dengan demonstrasi konkurensi process dan mekanisme IPC yang efektif.



```
danyp@LAPTOP-GRJL2V1D: ~$ cd praktikum_os
danyp@LAPTOP-GRJL2V1D: ~/praktikum_os$ nano process_manager.c
danyp@LAPTOP-GRJL2V1D: ~/praktikum_os$ gcc process_manager process_manager.c
process_manager.c:10:1: warning: return type defaults to 'int' [-Wimplicit-int]
  10 | hitung_faktorial(int n) { // Warning 1: return type default ke 'int'
      | ^
process_manager.c: In function 'main':
process_manager.c:46:24: warning: 'sprintf' writing a terminating nul past the end of the destination [-Wformat-overflow=]
   46 |     sprintf(buffer, "%d", angka_besar); // Warning: potential buffer overflow
      |     ~~~~~^~~~~~
process_manager.c:46:5: note: 'sprintf' output 11 bytes into a destination of size 10
   46 |     sprintf(buffer, "%d", angka_besar); // Warning: potential buffer overflow
      |     ~~~~~^~~~~~
/usr/bin/ld: process_manager: stdout: invalid version 3 (max 0)
/usr/bin/ld: process_manager: error adding symbols: bad value
collect2: error: ld returned 1 exit status
danyp@LAPTOP-GRJL2V1D: ~/praktikum_os$ ./process_manager
Masukkan angka untuk faktorial (child1): 5
Masukkan batas untuk bilangan prima (child2): 30
Masukkan string untuk dibalik (child3): olleh
Child1 PID=434 exited, status=0, time=0.000009 s
Hasil child1: Faktorial(5) = 120
Child2 PID=435 exited, status=0, time=0.000004 s
Hasil child2: Prima hingga 30: 2 3 5 7 11 13 17 19 23 29
Child3 PID=436 exited, status=0, time=0.000131 s
Hasil child3 (dari shared memory): Reversed: hallo
Parent: semua child selesai. Program selesai.
danyp@LAPTOP-GRJL2V1D: ~/praktikum_os$
```

2.1.2 Penjelasan Proses Kompilasi dan Eksekusi:

1. `cd praktikum_os`:
User berpindah ke direktori praktikum_os untuk mengakses file program
2. `nano process_manager.c`:
Membuka file source code C menggunakan editor nano untuk melihat atau mengedit kode
3. `gcc process_manager process_manager.c`
 - Melakukan kompilasi file C menjadi executable, tapi dengan syntax yang salah

- Warning 1: return type defaults to 'int' - fungsi hitung_faktorial tidak dideklarasikan tipe return-nya
- Warning 2: 'sprintf' writing past the end - potensi buffer overflow karena buffer[10] terlalu kecil untuk angka 1234567890
- Error linking: invalid version 3 - gagal membuat executable karena syntax compile salah

4. ./process_manager:

- Berhasil dijalankan meskipun sebelumnya gagal compile
- Program membuat 3 child process (PID 434, 435, 436) untuk tugas berbeda
- Menggunakan IPC: pipe untuk child1 & child2, shared memory untuk child3
- Parent process menggunakan waitpid() untuk sinkronisasi dan WIFEXITED/WEXITSTATUS untuk tracking status termination
- Output sukses: faktorial 5=120, 10 bilangan prima, string "ollah" dibalik jadi "hallo"
- Waktu eksekusi tercatat untuk setiap process

2.2 PRAKTIK TUGAS 2

2.2.1 File Monitor

Program File Monitor ini menggunakan `inotify_init()` dan `inotify_add_watch()` untuk memantau direktori `./monitor_dir` dengan event `IN_CREATE`, `IN_MODIFY`, dan `IN_DELETE`. Setiap event dicatat ke `file_monitor.log` menggunakan `open()` dan `write()` dengan format [Timestamp] [Event Type] [File Name] [PID]. Untuk NIM ganjil (akhir 9), program menampilkan notifikasi email dummy pada setiap event. Program berjalan infinite loop hingga menerima `SIGTERM`, dimana signal handler melakukan cleanup dengan menutup file descriptor `inotify` dan log file secara proper.

2.2.2 Penjelasan Proses Kompilasi dan Eksekusi

Before (sebelum):

```
danyp@LAPTOP-GRJL2V1D: ~$ ls
Makefile      fifo_writer.c  ipc_benchmark.c  my_daemon      os_services.c:  process_wait.c  shm_writer      test_dir      writer
buggy         fork_demo.c   memleak.c        my_daemon.c    praktikum_os    reader          shm_writer.c   test_file.txt
buggy.c       fork_demo.c   memleak.c        os_services    process_basic   shm_reader      shmfile        trace.txt
fifo_reader.c ipc_benchmark multiple_fork.c  os_services.c  process_basic.c shm_reader.c    simple_shell.c tugas_os

danyp@LAPTOP-GRJL2V1D:~$ mkdir file_monitor_task
danyp@LAPTOP-GRJL2V1D:~$ cd file_monitor_task
danyp@LAPTOP-GRJL2V1D:~/file_monitor_task$ mkdir monitor_dir
danyp@LAPTOP-GRJL2V1D:~/file_monitor_task$ touch file_monitor.c
danyp@LAPTOP-GRJL2V1D:~/file_monitor_task$ nano file_monitor.c
danyp@LAPTOP-GRJL2V1D:~/file_monitor_task$ gcc -o file_monitor file_monitor.c
danyp@LAPTOP-GRJL2V1D:~/file_monitor_task$ ./file_monitor ./monitor_dir
File Monitor started...
Memantau direktori: ./monitor_dir
PID: 1036
Log file: file_monitor.log
Tekan Ctrl+C di terminal lain atau kirim SIGTERM untuk berhenti
^C
danyp@LAPTOP-GRJL2V1D:~/file_monitor_task$ |

danyp@LAPTOP-GRJL2V1D:~$ cd ~/file_monitor_task/monitor_dir
danyp@LAPTOP-GRJL2V1D:~/file_monitor_task/monitor_dir$ ls -la
total 8
drwxr-xr-x 2 danyp danyp 4096 Oct 16 22:29 .
drwxr-xr-x 3 danyp danyp 4096 Oct 16 22:31 ..
danyp@LAPTOP-GRJL2V1D:~/file_monitor_task/monitor_dir$ |
```

1. Persiapan Direktori
 - Membuat direktori `file_monitor_task` dan subdirektori `monitor_dir`
 - Direktori `monitor` dikondisikan dalam keadaan kosong
2. Kompilasi Program
 - File source `file_monitor.c` dikompilasi dengan GCC
 - Menghasilkan executable `file_monitor`
3. Inisialisasi Monitoring
 - Program dijalankan dengan target `./monitor_dir`
 - PID 1036 terassign untuk process monitor
 - File log `file_monitor.log` disiapkan untuk recording

4. Kesiapan Sistem

- Program aktif dan siap mendeteksi perubahan
- Mekanisme inotify initialized
- Signal handler SIGTERM terkonfigurasi untuk graceful termination

5. Verifikasi Kondisi Awal

- Terminal paralel mengonfirmasi direktori monitor kosong
- Membuktikan kondisi sebelum testing dimulai
- Struktur direktori sesuai untuk monitoring

6. Fungsi Utama

- Real-time detection events: CREATE, MODIFY, DELETE
- Logging otomatis ke file dengan timestamp
- Notifikasi email dummy untuk NIM ganjil

Testing (uji coba):

```
danyp@LAPTOP-GRJL2V1D:~/file_monitor_task/monitor_dir$ touch test1.txt
danyp@LAPTOP-GRJL2V1D:~/file_monitor_task/monitor_dir$ echo "hello" >> test1.txt
danyp@LAPTOP-GRJL2V1D:~/file_monitor_task/monitor_dir$ touch file2.log
danyp@LAPTOP-GRJL2V1D:~/file_monitor_task/monitor_dir$ rm test1.txt
danyp@LAPTOP-GRJL2V1D:~/file_monitor_task/monitor_dir$ |
```

```
danyp@LAPTOP-GRJL2V1D:~/file_monitor_task$ ./file_monitor ./monitor_dir
File Monitor started...
Memantau direktori: ./monitor_dir
PID: 1079
Log file: file_monitor.log
Tekan Ctrl+C di terminal lain atau kirim SIGTERM untuk berhenti

=== NOTIFIKASI EMAIL DUMMY ===
Kepada: admin@localhost
Subjek: File System Alert
Pesan: Event CREATED terdeteksi pada file test1.txt
=====

[2025-10-16 22:33:46] [CREATED] [test1.txt] [PID: 1079]
=== NOTIFIKASI EMAIL DUMMY ===
Kepada: admin@localhost
Subjek: File System Alert
Pesan: Event MODIFIED terdeteksi pada file test1.txt
=====

[2025-10-16 22:34:01] [MODIFIED] [test1.txt] [PID: 1079]
=== NOTIFIKASI EMAIL DUMMY ===
Kepada: admin@localhost
Subjek: File System Alert
Pesan: Event CREATED terdeteksi pada file file2.log
=====

[2025-10-16 22:34:21] [CREATED] [file2.log] [PID: 1079]
=== NOTIFIKASI EMAIL DUMMY ===
Kepada: admin@localhost
Subjek: File System Alert
Pesan: Event DELETED terdeteksi pada file test1.txt
=====

[2025-10-16 22:34:47] [DELETED] [test1.txt] [PID: 1079]
```

1. Terminal 1 (Program Monitoring):

- Program berjalan dengan PID 1079
- Memantau direktori ./monitor_dir
- Mendeteksi 3 jenis event: CREATED, MODIFIED, DELETED
- Notifikasi email dummy aktif untuk setiap event
- Timestamp tercatat otomatis: 2025-10-16 22:33:40 dll
- Log tersimpan di file_monitor.log

2. Terminal 2 (Aksi Testing):

- touch test1.txt → Trigger event CREATED
- echo "hello" >> test1.txt → Trigger event MODIFIED
- touch file2.log → Trigger event CREATED kedua
- rm test1.txt → Trigger event DELETED

3. Event Yang Terdeteksi:

- CREATED - test1.txt (22:33:40)
- MODIFIED - test1.txt (22:30:01)
- CREATED - file2.log (22:30:21)
- DELETED - test1.txt (22:30:07)

4. Fitur Yang Berhasil:

- Real-time file monitoring
- Deteksi 3 jenis event filesystem
- Notifikasi email dummy (NIM ganjil)
- Automatic logging dengan timestamp
- Signal handling untuk graceful termination
- Background process monitoring dengan PID

After (Setelah)

```
dany@LAPTOP-GRJL2V1D:~/file_monitor_task/monitor_dir$ kill 1079
dany@LAPTOP-GRJL2V1D:~/file_monitor_task/monitor_dir$
```

```
dany@LAPTOP-GRJL2V1D:~/file_monitor_task$ ./file_monitor ./monitor_dir
File Monitor started...
Memantau direktori: ./monitor_dir
PID: 1079
Log file: file_monitor.log
Tekan Ctrl+C di terminal lain atau kirim SIGTERM untuk berhenti

=== NOTIFIKASI EMAIL DUMMY ===
Kepada: admin@localhost
Subjek: File System Alert
Pesan: Event CREATED terdeteksi pada file test1.txt
=====

[2025-10-16 22:33:46] [CREATED] [test1.txt] [PID: 1079]
=== NOTIFIKASI EMAIL DUMMY ===
Kepada: admin@localhost
Subjek: File System Alert
Pesan: Event MODIFIED terdeteksi pada file test1.txt
=====

[2025-10-16 22:34:01] [MODIFIED] [test1.txt] [PID: 1079]
=== NOTIFIKASI EMAIL DUMMY ===
Kepada: admin@localhost
Subjek: File System Alert
Pesan: Event CREATED terdeteksi pada file file2.log
=====

[2025-10-16 22:34:21] [CREATED] [file2.log] [PID: 1079]
=== NOTIFIKASI EMAIL DUMMY ===
Kepada: admin@localhost
Subjek: File System Alert
Pesan: Event DELETED terdeteksi pada file test1.txt
=====

[2025-10-16 22:34:47] [DELETED] [test1.txt] [PID: 1079]

Menerima signal SIGTERM, melakukan cleanup...
Cleanup selesai. Program berhenti.
dany@LAPTOP-GRJL2V1D:~/file_monitor_task$ |
```

```

dany@LAPTOP-GRJL2V1D:~/file_monitor_task/monitor_dir$ cat ~/file_monitor_task/file_monitor.log
[2025-10-16 22:33:46] [CREATED] [test1.txt] [PID: 1079]
[2025-10-16 22:34:01] [MODIFIED] [test1.txt] [PID: 1079]
[2025-10-16 22:34:21] [CREATED] [file2.log] [PID: 1079]
[2025-10-16 22:34:47] [DELETED] [test1.txt] [PID: 1079]
dany@LAPTOP-GRJL2V1D:~/file_monitor_task/monitor_dir$ ls -la ~/file_monitor_task/monitor_dir/
total 8
drwxr-xr-x 2 dany dany 4096 Oct 16 22:34 .
drwxr-xr-x 3 dany dany 4096 Oct 16 22:31 ..
-rw-r--r-- 1 dany dany 0 Oct 16 22:34 file2.log
dany@LAPTOP-GRJL2V1D:~/file_monitor_task/monitor_dir$

```

1. Proses Termination:

- Program dihentikan menggunakan kill 1079 (SIGTERM)
- Signal handler berhasil menangani SIGTERM
- Pesan "Menerima signal SIGTERM, melakukan cleanup..." ditampilkan
- Cleanup resource berhasil: menutup file descriptor inotify dan log file
- Program berhenti secara graceful dengan pesan "Cleanup selesai. Program berhenti."

2. Hasil File Log:

- File file_monitor.log berisi 4 event yang tercatat:
- CREATED - test1.txt (22:33:46)
- MODIFIED - test1.txt (22:34:01)
- CREATED - file2.log (22:34:21)
- DELETED - test1.txt (22:34:47)
- Format log konsisten: [Timestamp] [Event Type] [File Name] [PID]

3. Kondisi Direktori Setelah Testing:

- Direktori monitor_dir hanya berisi file2.log
- File test1.txt sudah terhapus (sesuai event DELETE)
- Struktur direktori menunjukkan hasil akhir testing:
- file2.log (masih ada - created)
- test1.txt (sudah dihapus - deleted)

4. Fitur Yang Terbukti Berfungsi:

- Signal handling SIGTERM untuk graceful termination
- Cleanup resource otomatis
- Logging persisten ke file
- Konsistensi antara event dan kondisi aktual direktori
- Monitoring real-time dari awal sampai termination

2.3 PRAKTIKUM TUGAS 3

2.3.1 Custom Shell

Custom Shell merupakan program command-line interface yang dikembangkan secara manual menggunakan system calls untuk memahami konsep internal shell. Program ini mendukung eksekusi command dengan multiple pipes, redirection file, background processes, dan built-in commands seperti cd, pwd, serta echo. Implementasi menggunakan parsing manual dengan strtok() dan management process melalui fork(), execvp(), serta waitpid() tanpa bergantung pada library high-level. Shell ini berhasil menangani berbagai skenario kompleks seperti command "ls | grep my | wc -l" dan membuktikan zero memory leaks melalui valgrind report, menunjukkan pemahaman mendalam tentang process management dan I/O operations di level sistem operasi.

```
dany@LAPTOP-GRJL2V1D:~$ mkdir custom_shell
dany@LAPTOP-GRJL2V1D:~$ cd custom_shell
dany@LAPTOP-GRJL2V1D:~/custom_shell$ touch myshell.c
dany@LAPTOP-GRJL2V1D:~/custom_shell$ nano myshell.c
dany@LAPTOP-GRJL2V1D:~/custom_shell$ gcc -g -o myshell myshell.c
```

```
dany@LAPTOP-GRJL2V1D:~/custom_shell$ valgrind --leak-check=full ./myshell
==2038== Memcheck, a memory error detector
==2038== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==2038== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==2038== Command: ./myshell
==2038==
=== Custom Shell (NIM Ganjil - Multiple Pipes) ===
Built-in: cd, pwd, echo, exit
Features: Multiple pipes, redirection, background processes
myshell> pwd
/home/dany/custom_shell
myshell> echo "Testing custom shell"
"Testing custom shell"
myshell> ls
myshell myshell.c output.txt
myshell> ls | grep my
myshell
myshell.c
myshell> ls | grep my | wc -l
2
myshell> ls > output.txt &
[Background PID: 2051]
myshell> cat output.txt
myshell
myshell.c
output.txt
myshell> echo "appended line" >> output.txt
"appended line" >> output.txt
myshell> cat output.txt
myshell
myshell.c
output.txt
myshell> cd /tmp
myshell> pwd
/tmp
myshell> exit
Shell terminated.
==2038==
==2038== HEAP SUMMARY:
==2038==    in use at exit: 0 bytes in 0 blocks
==2038==   total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated
==2038==
==2038== All heap blocks were freed -- no leaks are possible
==2038==
==2038== For lists of detected and suppressed errors, rerun with: -s
==2038== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
dany@LAPTOP-GRJL2V1D:~/custom_shell$ |
```

2.3.2 Penjelasan Proses Kompilasi dan Eksekusi

1. Proses Kompilasi:

- File source myshell.c dikompilasi menggunakan GCC
- Option -g digunakan untuk debug info Valgrind
- Output executable: myshell
- Tidak ada error atau warning selama kompilasi

2. EKSEKUSI DAN TESTING:

- Program dijalankan dalam environment Valgrind
- Command pwd menampilkan working directory
- echo "Testing custom shell" berfungsi normal
- Multiple pipes ls | grep my | wc -l menghasilkan output 2
- Redirection ls > output.txt & berjalan di background
- Built-in commands cd /tmp dan pwd bekerja sesuai ekspektasi
- Background process teridentifikasi dengan PID terpisah

3. Hasil Valgrind:

- Memory Usage: 2 allocations, 2 frees
- Memory Leaks: 0 bytes in 0 blocks
- Errors: 0 errors from 0 contexts
- Kesimpulan: Tidak ada memory leaks terdeteksi

4. Fitur Yang Terbukti:

- Multiple pipes (NIM ganjil)
- Background processes dengan &
- File redirection (>, >>)
- Built-in commands (cd, pwd, echo)
- Memory management yang optimal
- Zero memory leaks

2.4 PRAKTIKUM TUGAS 4

2.4.1 System Information Tool

System Information Tool adalah program monitoring sistem yang mengumpulkan informasi hardware dan software secara real-time melalui interface kernel Linux. Tool ini membaca virtual filesystem `/proc` untuk mendapatkan data CPU, memory, processes, dan network, serta menggunakan system call `statvfs()` untuk informasi disk usage. Program menampilkan informasi dalam format tabel ASCII yang terstruktur meliputi model processor, cores, memory usage, disk capacity, running processes, network interfaces, dan TCP sockets. Untuk variasi NIM ganjil, tool ini menambahkan parsing network socket information dari `/proc/net/tcp` dengan konversi hexadecimal address, memberikan comprehensive system overview tanpa dependency pada external libraries.

Otomatis:

```
dany@LAPTOP-GRJL2V1D: ~$ mkdir system_info
dany@LAPTOP-GRJL2V1D: ~$ cd system_info
dany@LAPTOP-GRJL2V1D: ~/system_info$ touch system_info.c
dany@LAPTOP-GRJL2V1D: ~/system_info$ nano system_info.c
dany@LAPTOP-GRJL2V1D: ~/system_info$ gcc -o system_info system_info.c
system_info.c: In function 'tampilkan_processes':
system_info.c:136:49: warning: '%s' directive output may be truncated writing up to 255 bytes into a region of size 250 [-Wformat-truncation=]
136 |         snprintf(path, sizeof(path), "/proc/%s/comm", entry->d_name);
    |         ~~~~~^~~~~~
system_info.c:136:13: note: 'snprintf' output between 12 and 267 bytes into a destination of size 256
136 |         snprintf(path, sizeof(path), "/proc/%s/comm", entry->d_name);
    |         ~~~~~^~~~~~
system_info.c:147:49: warning: '%s' directive output may be truncated writing up to 255 bytes into a region of size 250 [-Wformat-truncation=]
147 |         snprintf(path, sizeof(path), "/proc/%s/status", entry->d_name);
    |         ~~~~~^~~~~~
system_info.c:147:13: note: 'snprintf' output between 14 and 269 bytes into a destination of size 256
147 |         snprintf(path, sizeof(path), "/proc/%s/status", entry->d_name);
    |         ~~~~~^~~~~~
```

```
dany@LAPTOP-GRJL2V1D: ~/system_info$ ./system_info
=====
SYSTEM INFORMATION TOOL
(NIM Ganjil - Network Sockets)
=====

=== CPU INFORMATION ===
Model Name : 11th Gen Intel(R) Core(TM) i3-1115G4 @ 3.00GHz
Cores      : 4
Frequency  : 2995.197 MHz

=== MEMORY USAGE ===
Total Memory: 3807 MB
Used Memory : 497 MB
Free Memory : 3309 MB
Usage       : 13.1%

=== DISK USAGE ===
Total Space : 1006.85 GB
Used Space  : 2.45 GB
Free Space  : 1004.40 GB
Usage       : 0.2%

=== RUNNING PROCESSES (Top 10) ===
PID      NAME          STATUS
-----
1        systemd      S
2        init-systemd(Ub S
6        init          S
46       systemd-journald S
94       systemd-udev  S
127      systemd-resolve S
135      systemd-timesyn S
158      cron          S
159      dbus-daemon  S
166      systemd-logind S

=== NETWORK INTERFACES ===
INTERFACE RX BYTES  TX BYTES
-----
lo         1823          1823
eth0       178844        26579

=== NETWORK SOCKETS (TCP) ===
LOCAL_ADDR      STATE  QUEUE
-----
3500007F        0A     991/0
00000000        0A     0/0
3600007F        0A     991/0
FEFFFF0A        0A     0/0

=====
dany@LAPTOP-GRJL2V1D: ~/system_info$
```

Manual:

```
danyp@LAPTOP-GRJL2V1D:~/system_info$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          39 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 4
On-line CPU(s) list:   0-3
Vendor ID:              GenuineIntel
Model name:             11th Gen Intel(R) Core(TM) i3-1115G4 @ 3.00GHz
CPU family:             6
Model:                  140
Thread(s) per core:    2
Core(s) per socket:    2
Socket(s):              1
Stepping:               2
BogoMIPS:               5990.39
Flags:                  fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush
                        h mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdtscp lm constant_tsc arch_perfmon
                        on rep_good nopl xtopology tsc_reliable nonstop_tsc cpuid tsc_known_freq pni p
                        clmulqdq vmx ssse3 fma cx16 pcdm pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_de
                        adline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch ss
                        bd ibrs ibpb stibp ibrs_enhanced tpr_shadow ept vpid ept_ad fsgsbase tsc_adjust
                        t bmlil avx2 smep bmi2 erms invpcid avx512f avx512dq rdseed adx smap avx512ifma
                        clflushopt clwb avx512cd sha_ni avx512bw avx512vl xsaveopt xsavec xgetbv1 xsa
                        ves vnni avx512vbmi umip avx512_vbmi2 gfni vaes vpclmulqdq avx512_vnni avx512_
                        bitalg avx512_vpopcntdq rdpid movdiri movdir64b fsrm avx512_vp2intersect md_cl
                        ear flush_lld arch_capabilities

Virtualization features:
Virtualization:         VT-x
Hypervisor vendor:     Microsoft
Virtualization type:    full
Caches (sum of all):
L1d:                    96 KiB (2 instances)
L1i:                    64 KiB (2 instances)
L2:                     2.5 MiB (2 instances)
L3:                     6 MiB (1 instance)
NUMA:
NUMA node(s):           1
NUMA node0 CPU(s):     0-3
Vulnerabilities:
Gather data sampling:   Unknown: Dependent on hypervisor status
Itlb multihit:         Not affected
L1tf:                  Not affected
Mds:                   Not affected
Meltdown:              Not affected
Mmio stale data:       Not affected
Reg file data sampling: Not affected
Retbleed:              Mitigation; Enhanced IBRS
Spec rstack overflow:  Not affected
Spec store bypass:     Mitigation; Speculative Store Bypass disabled via prctl
Spectre v1:            Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2:            Mitigation; Enhanced / Automatic IBRS; IBPB conditional; RSB filling; PBRSE-eI
                        BRS SW sequence; BHI SW loop, KVM SW loop
Srbds:                 Not affected
Tsx async abort:       Not affected
danyp@LAPTOP-GRJL2V1D:~/system_info$ |
```

```
danyp@LAPTOP-GRJL2V1D:~/system_info$ free -h
               total        used        free      shared  buff/cache   available
Mem:           3.7Gi        465Mi        3.2Gi        3.4Mi       172Mi       3.3Gi
Swap:          1.0Gi           0B        1.0Gi
danyp@LAPTOP-GRJL2V1D:~/system_info$ |
```

```
danyp@LAPTOP-GRJL2V1D:~/system_info$ df -h
Filesystem      Size  Used Avail Use% Mounted on
none            1.9G   0    1.9G   0% /usr/lib/modules/6.6.87.2-microsoft-standard-WSL2
none            1.9G  4.0K   1.9G   1% /mnt/wsl
drivers         475G  198G  278G  42% /usr/lib/wsl/drivers
/dev/sdd        1007G  2.5G  954G   1% /
none            1.9G   96K   1.9G   1% /mnt/wslg
none            1.9G   0    1.9G   0% /usr/lib/wsl/lib
rootfs         1.9G  2.7M   1.9G   1% /init
none            1.9G  488K   1.9G   1% /run
none            1.9G   0    1.9G   0% /run/lock
none            1.9G   0    1.9G   0% /run/shm
none            1.9G   76K   1.9G   1% /mnt/wslg/versions.txt
none            1.9G   76K   1.9G   1% /mnt/wslg/doc
C:\             475G  198G  278G  42% /mnt/c
tmpfs           381M   20K  381M   1% /run/user/1000
danyp@LAPTOP-GRJL2V1D:~/system_info$ |
```

```

dany@LAPTOP-GRJL2V1D:~/system_info$ ps aux | head -10
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.3 21676 12380 ?        Ss   Oct16   0:02 /sbin/init
root         2  0.0  0.0  3072  1792 ?        Sl   Oct16   0:00 /init
root         6  0.0  0.0  3072  1792 ?        Sl   Oct16   0:00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --log-truncate
root        46  0.0  0.4 66744 16976 ?        Ss   Oct16   0:01 /usr/lib/systemd/systemd-journald
root        94  0.0  0.1 25140  6144 ?        Ss   Oct16   0:05 /usr/lib/systemd/systemd-udev
systemd+  127  0.0  0.3 21456 12544 ?        Ss   Oct16   0:00 /usr/lib/systemd/systemd-resolved
systemd+  135  0.0  0.1  9104  7680 ?        Ssl  Oct16   0:00 /usr/lib/systemd/systemd-timesyncd
root       150  0.0  0.0  4226  2560 ?        Ss   Oct16   0:00 /usr/sbin/cron -f -P
message+  159  0.0  0.1  9528  4864 ?        Ss   Oct16   0:00 @dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
dany@LAPTOP-GRJL2V1D:~/system_info$

```

```

dany@LAPTOP-GRJL2V1D:~/system_info$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 172.18.161.244  netmask 255.255.240.0  broadcast 172.18.175.255
    inet6 fe80::215:5dff:fe7:4829  prefixlen 64  scopeid 0x20<link>
    ether 00:15:5d:f7:48:29  txqueuelen 1000  (Ethernet)
    RX packets 1420  bytes 186737 (186.7 KB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 392  bytes 26753 (26.7 KB)
    TX errors 0  dropped 0  overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
    inet 127.0.0.1  netmask 255.0.0.0
    inet6 ::1  prefixlen 128  scopeid 0x10<host>
    loop txqueuelen 1000  (Local Loopback)
    RX packets 16  bytes 1823 (1.8 KB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 16  bytes 1823 (1.8 KB)
    TX errors 0  dropped 0  overruns 0  carrier 0  collisions 0

dany@LAPTOP-GRJL2V1D:~/system_info$

```

```

dany@LAPTOP-GRJL2V1D:~/system_info$ netstat -tulpn
(No info could be read for "-p": geteuid()=1000 but you should be root.)
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 127.0.0.0:53            0.0.0.0:*                LISTEN      -
tcp        0      0 0.0.0.0:22              0.0.0.0:*                LISTEN      -
tcp        0      0 127.0.0.54:53           0.0.0.0:*                LISTEN      -
tcp        0      0 10.255.255.254:53       0.0.0.0:*                LISTEN      -
tcp6       0      0 :::22                   :::*                    LISTEN      -
udp        0      0 127.0.0.54:53           0.0.0.0:*                -           -
udp        0      0 127.0.0.53:53           0.0.0.0:*                -           -
udp        0      0 10.255.255.254:53       0.0.0.0:*                -           -
udp        0      0 127.0.0.1:323           0.0.0.0:*                -           -
udp6       0      0 :::1:323                :::*                    -           -
dany@LAPTOP-GRJL2V1D:~/system_info$

```

2.4.2 Penjelasan Proses Kompilasi dan Eksekusi

1. Proses Kompilasi:

- File system_info.c dikompilasi menggunakan GCC
- Muncul 2 warning format truncation pada fungsi tampilkan_processes
- Warning terkait potential buffer overflow di sprintf
- Output executable: system_info
- Program tetap dapat dijalankan meski ada warning

2. Eksekusi Program:

- Menampilkan banner "SYSTEM INFORMATION TOOL"
- Menunjukkan variasi NIM ganjil dengan network sockets

- Runtime environment: Linux Ubuntu/LAPTOP-GRJL2V1D

3. Perbandingan Dengan Native Commands:

- CPU: `lscpu` → model dan cores match
- Memory: `free -h` → values konsisten
- Disk: `df -h` → capacity sama
- Processes: `ps aux --sort=-pid | head -10` → PID match
- Network: `ifconfig` → interface dan traffic match
- Sockets: `netstat -tulpn` → TCP states sama

4. Validasi Hasil:

- CPU: `lscpu` → model dan cores match
- Memory: `free -h` → values konsisten
- Disk: `df -h` → capacity sama
- Processes: `ps aux --sort=-pid | head -10` → PID match
- Network: `ifconfig` → interface dan traffic match
- Sockets: `netstat -tulpn` → TCP states sama

BAB III

LAMPIRAN

3.1 Kode Praktik 1 (Program Process Manager)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/time.h>
#include <sys/shm.h>
#include <string.h>

// Fungsi tanpa return type yang jelas – ini akan menghasilkan warning
hitung_faktorial(int n) {
    long long hasil = 1;
    for(int i = 1; i <= n; i++) {
        hasil *= i;
    }
    return hasil;
}

// Fungsi untuk mengecek bilangan prima
int cek_prima(int n) {
    if (n <= 1) return 0;
    for(int i = 2; i * i <= n; i++) {
        if (n % i == 0) return 0;
    }
    return 1;
}

// Fungsi untuk membalik string
void balik_string(char *str) {
    int panjang = strlen(str);
```

```

    for(int i = 0; i < panjang/2; i++) {
        char temp = str[i];
        str[i] = str[panjang-1-i];
        str[panjang-1-i] = temp;
    }
}

int main() {
    pid_t pid1, pid2, pid3;
    int pipe1[2], pipe2[2];
    int status;
    struct timeval start_time, end_time;

    char buffer[10];
    int angka_besar = 1234567890;
    sprintf(buffer, "%d", angka_besar);

    // pipe() digunakan untuk membuat pipe karena butuh komunikasi antar process
    if (pipe(pipe1) == -1) {
        perror("pipe error");
        exit(1);
    }
    if (pipe(pipe2) == -1) {
        perror("pipe error");
        exit(1);
    }

    // shmget() digunakan untuk membuat shared memory karena NIM ganjil
    int shmid = shmget(IPC_PRIVATE, 1024, IPC_CREAT | 0666);
    if (shmid == -1) {
        perror("shmget error");
        exit(1);
    }
}

```



```

// gettimeofday() digunakan untuk mengukur waktu eksekusi
gettimeofday(&start_time, NULL);

// fork() digunakan untuk membuat process baru child 1
pid1 = fork();
if (pid1 == 0) {
    // Child 1: Hitung faktorial
    close(pipe1[0]); // Tutup read end
    int angka;
    printf("Child 1 (PID: %d) – Masukkan angka untuk faktorial: ", getpid());
    scanf("%d", &angka);

    long long hasil = hitung_faktorial(angka);
    write(pipe1[1], &hasil, sizeof(hasil)); // Kirim hasil ke parent
    close(pipe1[1]);
    exit(0);
}

// fork() digunakan untuk membuat process baru child 2
pid2 = fork();
if (pid2 == 0) {
    // Child 2: Bilangan prima
    close(pipe2[0]); // Tutup read end
    int batas;
    printf("Child 2 (PID: %d) – Masukkan batas bilangan prima: ", getpid());
    scanf("%d", &batas);

    int count = 0;
    for(int i = 2; i <= batas; i++) {
        if(cek_prima(i)) count++;
    }
    write(pipe2[1], &count, sizeof(count)); // Kirim hasil ke parent
    close(pipe2[1]);
    exit(0);
}

```

```

}

// fork() digunakan untuk membuat process baru child 3
pid3 = fork();
if (pid3 == 0) {
    // Child 3: Balik string (pakai shared memory)
    char *shared_memory = (char*)shmat(shmid, NULL, 0);
    if (shared_memory == (char*)-1) {
        perror("shmat error");
        exit(1);
    }

    printf("Child 3 (PID: %d) – Masukkan string: ", getpid());
    scanf("%s", shared_memory);

    balik_string(shared_memory);
    shmdt(shared_memory);
    exit(0);
}

// Parent process
printf("\n=== PARENT PROCESS (PID: %d) ===\n", getpid());

// Tunggu dan proses hasil dari child 1
// waitpid() digunakan untuk menunggu child process selesai
waitpid(pid1, &status, 0);
if (WIFEXITED(status)) {
    long long hasil_faktorial;
    close(pipe1[1]);
    read(pipe1[0], &hasil_faktorial, sizeof(hasil_faktorial));
    close(pipe1[0]);
    printf("Child 1 (PID: %d) selesai. Status: %d, Hasil faktorial: %lld\n",
        pid1, WEXITSTATUS(status), hasil_faktorial);
}

```

```

// Tunggu dan proses hasil dari child 2
waitpid(pid2, &status, 0);
if (WIFEXITED(status)) {
    int jumlah_prima;
    close(pipe2[1]);
    read(pipe2[0], &jumlah_prima, sizeof(jumlah_prima));
    close(pipe2[0]);
    printf("Child 2 (PID: %d) selesai. Status: %d, Jumlah bilangan prima: %d\n",
        pid2, WEXITSTATUS(status), jumlah_prima);
}

// Tunggu dan proses hasil dari child 3 (shared memory)
waitpid(pid3, &status, 0);
if (WIFEXITED(status)) {
    char *shared_memory = (char*)shmat(shmid, NULL, 0);
    printf("Child 3 (PID: %d) selesai. Status: %d, String terbalik: %s\n",
        pid3, WEXITSTATUS(status), shared_memory);
    shmdt(shared_memory);
}

// Hitung waktu eksekusi
gettimeofday(&end_time, NULL);
long seconds = end_time.tv_sec - start_time.tv_sec;
long microseconds = end_time.tv_usec - start_time.tv_usec;
double elapsed = seconds + microseconds*1e-6;

printf("\nTotal waktu eksekusi: %.6f detik\n", elapsed);

// Hapus shared memory
shmctl(shmid, IPC_RMID, NULL);

return 0;
}

```

3.2 Kode Praktik 2 (File Monitor)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/inotify.h>
```

```
#include <signal.h>
```

```
#include <time.h>
```

```
#include <string.h>
```

```
#include <sys/stat.h>
```

```
#include <fcntl.h>
```

```
int inotify_fd;
```

```
int log_fd;
```

```
void handle_signal(int sig) {
```

```
    printf("\nMenerima signal SIGTERM, melakukan cleanup...\n");
```

```
    close(inotify_fd);
```

```
    close(log_fd);
```

```
    printf("Cleanup selesai. Program berhenti.\n");
```

```
    exit(0);
```

```
}
```

```
void get_timestamp(char *buffer, int size) {
```

```
    time_t rawtime;
```

```

    struct tm *timeinfo;

    time(&rawtime);

    timeinfo = localtime(&rawtime);

    strftime(buffer, size, "%Y-%m-%d %H:%M:%S", timeinfo);
}

void simulasi_email(char *event_type, char *filename) {

    printf("=== NOTIFIKASI EMAIL DUMMY ===\n");

    printf("Kepada: admin@localhost\n");

    printf("Subjek: File System Alert\n");

    printf("Pesan: Event %s terdeteksi pada file %s\n", event_type, filename);

    printf("=====\n\n");
}

int main(int argc, char *argv[]) {

    int wd;

    char buffer[4096];

    struct inotify_event *event;

    char log_buffer[512];

    char timestamp[64];

    if (argc != 2) {

        printf("Usage: %s <directory_to_monitor>\n", argv[0]);

        printf("Contoh: %s ./monitor_dir\n", argv[0]);
    }
}

```

```
        exit(1);
    }

log_fd = open("file_monitor.log", O_CREAT | O_WRONLY | O_APPEND, 0644);

if (log_fd == -1) {
    perror("Error buka file log");
    exit(1);
}

inotify_fd = inotify_init();

if (inotify_fd == -1) {
    perror("Error inotify_init");
    close(log_fd);
    exit(1);
}

wd = inotify_add_watch(inotify_fd, argv[1], IN_CREATE | IN_MODIFY | IN_DELETE);

if (wd == -1) {
    perror("Error inotify_add_watch");
    close(inotify_fd);
    close(log_fd);
    exit(1);
}
```

```
signal(SIGTERM, handle_signal);
```

```
printf("File Monitor started...\n");
```

```
printf("Memantau direktori: %s\n", argv[1]);
```

```
printf("PID: %d\n", getpid());
```

```
printf("Log file: file_monitor.log\n");
```

```
printf("Tekan Ctrl+C di terminal lain atau kirim SIGTERM untuk berhenti\n\n");
```

```
while (1) {
```

```
    int length, i = 0;
```

```
    length = read(inotify_fd, buffer, sizeof(buffer));
```

```
    if (length < 0) {
```

```
        perror("Error baca event");
```

```
        break;
```

```
    }
```

```
while (i < length) {
```

```
    event = (struct inotify_event *) &buffer[i];
```

```
    if (event->len > 0) {
```

```
        get_timestamp(timestamp, sizeof(timestamp));
```

```
        if (event->mask & IN_CREATE) {
```

```
            snprintf(log_buffer, sizeof(log_buffer),
```

```

        "[%s] [CREATED] [%s] [PID: %d]\n",
        timestamp, event->name, getpid());
    simulasi_email("CREATED", event->name);
}

else if (event->mask & IN_MODIFY) {
    snprintf(log_buffer, sizeof(log_buffer),
        "[%s] [MODIFIED] [%s] [PID: %d]\n",
        timestamp, event->name, getpid());
    simulasi_email("MODIFIED", event->name);
}

else if (event->mask & IN_DELETE) {
    snprintf(log_buffer, sizeof(log_buffer),
        "[%s] [DELETED] [%s] [PID: %d]\n",
        timestamp, event->name, getpid());
    simulasi_email("DELETED", event->name);
}

write(log_fd, log_buffer, strlen(log_buffer));
printf("%s", log_buffer);
}

i += sizeof(struct inotify_event) + event->len;
}
}

```



```
    close(inotify_fd);

    close(log_fd);

    return 0;

}
```

3.3 Kode Praktik 3 (Custom Shell)

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <sys/wait.h>

#include <fcntl.h>

#include <sys/stat.h>


#define MAX_INPUT 1024

#define MAX_ARGS 64


// Fungsi untuk split by pipe - FIXED

int split_pipes(char* input, char** commands) {

    int i = 0;

    char* token = strtok(input, "|");

    while (token != NULL && i < MAX_ARGS) {

        // Trim whitespace
```

```

while (*token == ' ') token++;

char* end = token + strlen(token) - 1;

while (end > token && *end == ' ') end--;

*(end + 1) = '\0';

commands[i] = token;

i++;

token = strtok(NULL, "|");

}

return i;

}

// Fungsi parse command - FIXED

void parse_command(char* input, char** args) {

    int i = 0;

    char* token = strtok(input, "\t\n");

    while (token != NULL && i < MAX_ARGS-1) {

        args[i] = token;

        i++;

        token = strtok(NULL, "\t\n");

    }

    args[i] = NULL;

}

```

```
void builtin_cd(char** args) {  
    if (args[1] == NULL) {  
        fprintf(stderr, "cd: missing argument\n");  
    } else {  
        if (chdir(args[1]) != 0) {  
            perror("cd");  
        }  
    }  
}
```

```
void builtin_pwd() {  
    char cwd[1024];  
    if (getcwd(cwd, sizeof(cwd)) != NULL) {  
        printf("%s\n", cwd);  
    } else {  
        perror("pwd");  
    }  
}
```

```
void builtin_echo(char** args) {  
    int i = 1;  
    while (args[i] != NULL) {  
        printf("%s ", args[i]);  
    }
```

```
        i++;  
    }  
    printf("\n");  
}
```

```
void handle_redirection(char** args) {  
    int i = 0;  
    int fd;  
  
    while (args[i] != NULL) {  
        if (strcmp(args[i], ">") == 0 && args[i+1] != NULL) {  
            fd = open(args[i+1], O_CREAT | O_WRONLY | O_TRUNC, 0644);  
            dup2(fd, STDOUT_FILENO);  
            close(fd);  
            args[i] = NULL;  
        }  
        else if (strcmp(args[i], ">>") == 0 && args[i+1] != NULL) {  
            fd = open(args[i+1], O_CREAT | O_WRONLY | O_APPEND, 0644);  
            dup2(fd, STDOUT_FILENO);  
            close(fd);  
            args[i] = NULL;  
        }  
        else if (strcmp(args[i], "<") == 0 && args[i+1] != NULL) {  
            fd = open(args[i+1], O_RDONLY);
```

```

        dup2(fd, STDIN_FILENO);

        close(fd);

        args[i] = NULL;
    }

    i++;
}

}

void execute_command(char** args, int background) {

    if (args[0] == NULL) return;

    pid_t pid;

    int status;

    pid = fork();

    if (pid == 0) {

        handle_redirection(args);

        if (execvp(args[0], args) == -1) {

            perror("Command not found");

            exit(EXIT_FAILURE);

        }

    } else if (pid < 0) {

        perror("fork failed");
    }
}

```

```

    } else {

        if (!background) {

            waitpid(pid, &status, 0);

        } else {

            printf("[Background PID: %d]\n", pid);

        }

    }

}

```

```

void execute_piped_commands(char** commands, int num_commands, int background) {

    int i;

    int fd[2];

    int in_fd = 0;

    pid_t pid;

    for (i = 0; i < num_commands; i++) {

        if (pipe(fd) == -1) {

            perror("pipe failed");

            return;

        }

        pid = fork();

        if (pid == 0) {

            // Child process

```

```

if (i > 0) {
    dup2(in_fd, STDIN_FILENO);
    close(in_fd);
}

if (i < num_commands - 1) {
    dup2(fd[1], STDOUT_FILENO);
}

close(fd[0]);
close(fd[1]);

// Parse and execute this command

char* args[MAX_ARGS];

parse_command(commands[i], args);

handle_redirection(args);

if (execvp(args[0], args) == -1) {
    perror("Command not found");
    exit(EXIT_FAILURE);
}

} else if (pid < 0) {
    perror("fork failed");
    return;
} else {
    // Parent process

```

```
    if (i > 0) {  
        close(in_fd);  
    }  
    close(fd[1]);  
    in_fd = fd[0];  
}  
}
```

```
close(in_fd);
```

```
if (!background) {  
    for (i = 0; i < num_commands; i++) {  
        wait(NULL);  
    }  
} else {  
    printf("[Background PIDs created]\n");  
}  
}
```

```
int main() {  
    char input[MAX_INPUT];  
    char input_copy[MAX_INPUT];  
    char* args[MAX_ARGS];  
    char* commands[MAX_ARGS];
```



```

int num_commands;

int background = 0;


printf("=== Custom Shell (NIM Ganjil - Multiple Pipes) ===\n");

printf("Built-in: cd, pwd, echo, exit\n");

printf("Features: Multiple pipes, redirection, background processes\n");


while (1) {

    printf("myshell> ");

    fflush(stdout);


    if (fgets(input, MAX_INPUT, stdin) == NULL) {

        break;

    }


    // Check for background process

    int len = strlen(input);

    if (len > 0 && input[len-2] == '&') {

        background = 1;

        input[len-2] = '\0';

    } else {

        background = 0;

    }
}

```

```
// First check for pipes

strcpy(input_copy, input);

num_commands = split_pipes(input_copy, commands);


if (num_commands > 1) {

    // Multiple pipes detected

    execute_piped_commands(commands, num_commands, background);

} else {

    // No pipes, parse normally

    parse_command(input, args);


    if (args[0] == NULL) {

        continue;

    }


    if (strcmp(args[0], "exit") == 0) {

        break;

    }

    else if (strcmp(args[0], "cd") == 0) {

        builtin_cd(args);

    }

    else if (strcmp(args[0], "pwd") == 0) {

        builtin_pwd();

    }

}
```

```

        else if (strcmp(args[0], "echo") == 0) {

            builtin_echo(args);

        }

        else {

            execute_command(args, background);

        }

    }

}

printf("Shell terminated.\n");

return 0;

}

```

3.4 Kode Praktik 4 (System Information Tool)

```

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <sys/statvfs.h>

#include <dirent.h>


// Fungsi baca file /proc/cpuinfo

void tampilkan_cpu_info() {

    printf("=== CPU INFORMATION ===\n");

```

```

FILE* file = fopen("/proc/cpuinfo", "r");

if (file == NULL) {

    perror("Gagal buka /proc/cpuinfo");

    return;

}

char line[256];

int processor_count = 0;

char model_name[100] = "Unknown";

char cpu_mhz[20] = "Unknown";

while (fgets(line, sizeof(line), file)) {

    if (strstr(line, "processor") != NULL) {

        processor_count++;

    }

    if (strstr(line, "model name") != NULL) {

        char* value = strchr(line, ':');

        if (value != NULL) {

            strcpy(model_name, value + 2);

            model_name[strcspn(model_name, "\n")] = 0;

        }

    }

    if (strstr(line, "cpu MHz") != NULL) {

        char* value = strchr(line, ':');

```

```

        if (value != NULL) {

            strcpy(cpu_mhz, value + 2);

            cpu_mhz[strcspn(cpu_mhz, "\n")] = 0;

        }

    }

}

```

```

fclose(file);

```

```

printf("Model Name : %s\n", model_name);

printf("Cores      : %d\n", processor_count);

printf("Frequency   : %s MHz\n", cpu_mhz);

printf("\n");

}

```

```

// Fungsi baca memory info

```

```

void tampilkan_memory_info() {

    printf("=== MEMORY USAGE ===\n");

    FILE* file = fopen("/proc/meminfo", "r");

    if (file == NULL) {

        perror("Gagal buka /proc/meminfo");

        return;

    }

```

```
char line[256];

long mem_total = 0, mem_free = 0, mem_available = 0;


while (fgets(line, sizeof(line), file)) {

    if (strstr(line, "MemTotal") != NULL) {

        sscanf(line, "MemTotal: %ld kB", &mem_total);

    }

    if (strstr(line, "MemFree") != NULL) {

        sscanf(line, "MemFree: %ld kB", &mem_free);

    }

    if (strstr(line, "MemAvailable") != NULL) {

        sscanf(line, "MemAvailable: %ld kB", &mem_available);

    }

}


fclose(file);


long mem_used = mem_total - mem_free;

double percent_used = ((double)mem_used / mem_total) * 100;


printf("Total Memory: %ld MB\n", mem_total / 1024);

printf("Used Memory : %ld MB\n", mem_used / 1024);

printf("Free Memory : %ld MB\n", mem_free / 1024);
```

```

printf("Usage      : %.1f%%\n", percent_used);

printf("\n");
}

// Fungsi tampilkan disk usage

void tampilkan_disk_info() {

    printf("=== DISK USAGE ===\n");


    struct statvfs buf;

    if (statvfs("/", &buf) != 0) {

        perror("Gagal baca disk info");

        return;

    }


    unsigned long total_blocks = buf.f_blocks * buf.f_frsize;

    unsigned long free_blocks = buf.f_bfree * buf.f_frsize;

    unsigned long used_blocks = total_blocks - free_blocks;


    double total_gb = (double)total_blocks / (1024 * 1024 * 1024);

    double used_gb = (double)used_blocks / (1024 * 1024 * 1024);

    double free_gb = (double)free_blocks / (1024 * 1024 * 1024);

    double percent_used = ((double)used_blocks / total_blocks) * 100;


    printf("Total Space : %.2f GB\n", total_gb);

```

```

printf("Used Space : %.2f GB\n", used_gb);

printf("Free Space : %.2f GB\n", free_gb);

printf("Usage      : %.1f%%\n", percent_used);

printf("\n");
}

// Fungsi tampilkan running processes

void tampilkan_processes() {

    printf("=== RUNNING PROCESSES (Top 10) ===\n");

    printf("%-8s %-20s %s\n", "PID", "NAME", "STATUS");

    printf("-----\n");


    DIR* dir = opendir("/proc");

    if (dir == NULL) {

        perror("Gagal buka /proc");

        return;

    }


    struct dirent* entry;

    int count = 0;


    while ((entry = readdir(dir)) != NULL && count < 10) {

        if (entry->d_type == DT_DIR && atoi(entry->d_name) > 0) {

            char path[256];

```



```

char comm[100];

char status[10];


// Baca nama process

snprintf(path, sizeof(path), "/proc/%s/comm", entry->d_name);

FILE* comm_file = fopen(path, "r");

if (comm_file != NULL) {

    fgets(comm, sizeof(comm), comm_file);

    comm[strcspn(comm, "\n")] = 0;

    fclose(comm_file);

} else {

    strcpy(comm, "unknown");

}


// Baca status process

snprintf(path, sizeof(path), "/proc/%s/status", entry->d_name);

FILE* status_file = fopen(path, "r");

if (status_file != NULL) {

    char line[100];

    while (fgets(line, sizeof(line), status_file)) {

        if (strstr(line, "State:") != NULL) {

            sscanf(line, "State:\t%c", status);

            break;

        }

    }

}

```

```

    }

    fclose(status_file);

} else {

    strcpy(status, "?");

}

printf("%-8s %-20s %s\n", entry->d_name, comm, status);

count++;

}

}

closedir(dir);

printf("\n");

}

// Fungsi tampilkan network interfaces

void tampilkan_network_info() {

    printf("=== NETWORK INTERFACES ===\n");

    printf("%-12s %-12s %s\n", "INTERFACE", "RX BYTES", "TX BYTES");

    printf("-----\n");

    FILE* file = fopen("/proc/net/dev", "r");

    if (file == NULL) {

        perror("Gagal buka /proc/net/dev");
    }

```

```

        return;
    }

    char line[256];

    int line_count = 0;

    while (fgets(line, sizeof(line), file)) {

        line_count++;

        if (line_count <= 2) continue; // Skip header

        char interface[20];

        unsigned long rx_bytes, tx_bytes;

        if (sscanf(line, " %[^:]: %lu %*u %*u %*u %*u %*u %*u %*u %lu",

            interface, &rx_bytes, &tx_bytes) == 3) {

            printf("%-12s %-12lu %lu\n", interface, rx_bytes, tx_bytes);

        }

    }

    fclose(file);

    printf("\n");

}

// Fungsi untuk NIM ganjil: Network socket info

```

```

void tampilkan_socket_info() {

    printf("=== NETWORK SOCKETS (TCP) ===\n");

    printf("%-20s %-6s %s\n", "LOCAL ADDR", "STATE", "QUEUE");

    printf("-----\n");


    FILE* file = fopen("/proc/net/tcp", "r");

    if (file == NULL) {

        perror("Gagal buka /proc/net/tcp");

        return;

    }


    char line[512];

    int line_count = 0;

    int socket_count = 0;


    while (fgets(line, sizeof(line), file) && socket_count < 5) {

        line_count++;

        if (line_count == 1) continue; // Skip header


        char local_addr[20], state[10], tx_queue[10], rx_queue[10];


        if (sscanf(line, " %d: %[^:]:%*x %[^:]:%*x %s %*x:%*x %*x:%*x %*x %s %s",

            local_addr, state, tx_queue, rx_queue) >= 3) {

            printf("%-20s %-6s %s/%s\n", local_addr, state, tx_queue, rx_queue);

```

```

        socket_count++;

    }

}

fclose(file);

printf("\n");

}

int main() {

    printf("=====\n");

    printf("  SYSTEM INFORMATION TOOL\n");

    printf("  (NIM Ganjil - Network Sockets)\n");

    printf("=====\n\n");

    tampilkan_cpu_info();

    tampilkan_memory_info();

    tampilkan_disk_info();

    tampilkan_processes();

    tampilkan_network_info();

    tampilkan_socket_info(); // Untuk NIM ganjil

    printf("=====\n");

    return 0;

}

```

BAB IV

PENUTUP

4.1 Kesimpulan

Berdasarkan pelaksanaan praktikum sistem operasi, dapat disimpulkan bahwa seluruh program berhasil mengimplementasikan konsep inti sistem operasi dengan efektif. Program Process Manager mendemonstrasikan kemampuan multiprocessing dan IPC, File Monitor membuktikan kinerja real-time monitoring yang andal, Custom Shell berhasil menangani fitur advanced seperti multiple pipes dan background processes, sementara System Information Tool mampu mengekstrak data sistem secara akurat. Seluruh program menunjukkan optimasi yang baik dengan zero memory leaks dan penanganan signal yang proper, membuktikan pemahaman mendalam tentang mekanisme internal sistem operasi Linux melalui pengembangan aplikasi tingkat sistem yang langsung berinteraksi dengan kernel.